

LAB 14: ROUTER NÂNG CAO & XÁC THỰC

Mục tiêu bài thực hành

1. **Nested Routes:** Xây dựng cấu trúc Layout chung (Navbar, Footer) chứa các trang con.
2. **Dynamic Routes:** Truy xuất tham số từ URL (ví dụ: id của sản phẩm) để hiển thị dữ liệu động.
3. **Private/Protected Routes:** Tích hợp Context API để quản lý trạng thái đăng nhập, bảo vệ các trang yêu cầu quyền truy cập.

Chuẩn bị dự án

Mở terminal và chạy các lệnh sau để khởi tạo dự án và cài đặt thư viện:

```
npx create-react-app react-advanced-routing
cd react-advanced-routing
npm install react-router-dom
```

Bước 1: Thiết lập Mock Authentication (Context API)

Tạo một Global State để lưu trữ trạng thái đăng nhập của người dùng. Trạng thái này sẽ quyết định việc cho phép truy cập vào Protected Routes hay không.

Tạo file: src/context/AuthContext.jsx

```
import React, { createContext, useState, useContext } from 'react';
// Khởi tạo Context
const AuthContext = createContext();
export const AuthProvider = ({ children }) => {
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const login = () => setIsAuthenticated(true);
  const logout = () => setIsAuthenticated(false);
  return (
    <AuthContext.Provider value={{ isAuthenticated, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};
// Custom hook để sử dụng AuthContext dễ dàng hơn
export const useAuth = () => useContext(AuthContext);
```

- **Giải thích:** AuthProvider cung cấp biến isAuthenticated và 2 hàm login, logout cho toàn bộ ứng dụng. Bất kỳ component nào nằm trong AuthProvider đều có thể gọi useAuth() để lấy dữ liệu này.

Bước 2: Thiết lập Layout chung & Nested Routes

Sử dụng <Outlet/> để tạo một khung giao diện chung (có Navbar cố định) và thay đổi nội dung bên trong dựa trên URL.

Tạo file: src/components/MainLayout.jsx

```
import React from 'react';
import { Link, Outlet } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
const MainLayout = () => {
  const { isAuthenticated, logout } = useAuth();
  return (
    <div style={{ fontFamily: 'Arial', padding: '20px' }}>
      <nav style={{ padding: '10px 0', border: '1px solid #ccc',
marginBottom: '20px' }}>
        <Link to="/" style={{ margin: '0 15px' }}>Trang chủ</Link>
        <Link to="/products" style={{ margin: '0 15px' }}>Sản phẩm</Link>
        <Link to="/dashboard" style={{ margin: '0 15px' }}>Dashboard
(Private)</Link>

        {isAuthenticated ? (
          <button onClick={logout} style={{ float: 'right' }}>
            Đăng xuất
          </button>
        ) : (
          <Link to="/login" style={{ float: 'right' }}>Đăng nhập</Link>
        )}
      </nav>
      <div style={{ clear: 'both' }}>
        <div>{/* Outlet là nơi các component con (Nested Routes) sẽ được render */}>
          <Outlet />
        </div>
      </div>
    </div>
  );
};
export default MainLayout;
```

- **Giải thích:** Khung layout có thanh điều hướng (Navbar). <Outlet/> chính là placeholder (chỗ trống) mà React Router sẽ nhúng các component con vào (như Home, Products, Dashboard) tương ứng với từng đường dẫn.

Bước 3: Dynamic Routes - Trang danh sách và chi tiết sản phẩm

Xây dựng trang danh sách sản phẩm và trang chi tiết nhận id động từ URL.

Tạo file: src/pages/Products.jsx

```
import React from 'react';
import { Link } from 'react-router-dom';
const Products = () => {
  const productList = [
    { id: 1, name: 'Khóa học ReactJS' },
    { id: 2, name: 'Khóa học NodeJS' },
    { id: 3, name: 'Khóa học C++' },
  ];
  return (
    <div>
      <h2>Danh sách sản phẩm</h2>
      <ul>
        {productList.map((prod) => (
          <li key={prod.id}>
            <Link to={` /products/${prod.id}`}>{prod.name}</Link>
          </li>
        ))}
      </ul>
    </div>
  );
};
export default Products;
```

Tạo file: src/pages/ProductDetail.jsx

```
import React from 'react';
import { useParams, useNavigate } from 'react-router-dom';
const ProductDetail = () => {
  // Trích xuất tham số 'id' từ URL động
  const { id } = useParams();
  const navigate = useNavigate();
  return (
    <div>
      <h2>Chi tiết sản phẩm</h2>
      <p>Đang hiển thị thông tin cho sản phẩm có ID: <strong>{id}</strong></p>

      <button onClick={() => navigate('/products')}>
        Quay lại danh sách
      </button>
    </div>
  );
};
```

```
);  
};  
export default ProductDetail;
```

- **Giải thích:**

- useParams(): Lấy giá trị :id trên thanh địa chỉ.
- useNavigate(): Điều hướng bằng code (chuyển người dùng về trang danh sách khi nhấn nút) thay vì dùng thẻ <Link>.

Bước 4: Chức năng Đăng nhập & Protected Route

Tạo ra một "vệ sĩ" (HOC - Higher Order Component) để kiểm tra quyền truy cập trước khi cho phép vào các trang nội bộ.

Tạo file: src/components/ProtectedRoute.jsx

```
import React from 'react';  
import { Navigate, useLocation } from 'react-router-dom';  
import { useAuth } from '../context/AuthContext';  
const ProtectedRoute = ({ children }) => {  
  const { isAuthenticated } = useAuth();  
  const location = useLocation();  
  if (!isAuthenticated) {  
    // Nếu chưa đăng nhập, chuyển hướng về trang login  
    // Lưu lại vị trí (location) người dùng muốn vào để redirect sau khi login xong  
    // (tùy chọn)  
    return <Navigate to="/login" state={{ from: location }} replace />;  
  }  
  // Nếu đã đăng nhập, cho phép render component con  
  return children;  
};  
export default ProtectedRoute;
```

Tạo file: src/pages/Login.jsx (Trang đăng nhập) và src/pages/Dashboard.jsx (Trang nội bộ)

```
// --- src/pages/Login.jsx ---  
import React from 'react';  
import { useNavigate, useLocation } from 'react-router-dom';  
import { useAuth } from '../context/AuthContext';  
const Login = () => {  
  const { login } = useAuth();  
  const navigate = useNavigate();  
  const location = useLocation();  
  // Lấy đường dẫn mà người dùng muốn truy cập trước khi bị chặn, mặc định là '/'  
  const from = location.state?.from?.pathname || '/';  
  // ...
```

```
const handleLogin = () => {
  login(); // Cập nhật state đăng nhập thành true
  navigate(from, { replace: true }); // Điều hướng trở lại trang trước đó
};
return (
  <div>
    <h2>Trang Đăng Nhập</h2>
    <p>Bạn cần đăng nhập để xem nội dung nội bộ.</p>
    <button onClick={handleLogin} style={{ padding: '10px', background: 'blue',
color: 'white' }}>
      Đăng nhập (Mock)
    </button>
  </div>
);
};
export default Login;
JavaScript
// --- src/pages/Dashboard.jsx ---
import React from 'react';
const Dashboard = () => {
  return (
    <div style={{ background: '#eef', padding: '20px' }}>
      <h2>Dashboard Nội Bộ</h2>
      <p>Chúc mừng! Bạn đã vượt qua chốt bảo vệ xác thực để vào đây.</p>
    </div>
  );
};
export default Dashboard;
```

- **Giải thích:** ProtectedRoute kiểm tra biến isAuthenticated. Nếu sai, dùng <Navigate> ép URL về /login. Tại trang Login, sau khi bấm đăng nhập, hệ thống sẽ gọi login() và tự động navigate đưa người dùng về lại trang họ định vào lúc đầu.

Bước 5: Tích hợp định tuyến tại App.js

Gắn kết toàn bộ các thành phần lại với nhau.

Mở file: src/App.js và thay thế bằng code sau:

```
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import MainLayout from '../components/MainLayout';
import ProtectedRoute from '../components/ProtectedRoute';
import Products from '../pages/Products';
```

LAB 14: ROUTER NÂNG CAO – XÁC THỰC

```
import ProductDetail from './pages/ProductDetail';
import Login from './pages/Login';
import Dashboard from './pages/Dashboard';
const Home = () => <h2>Trang chủ (Public)</h2>;
const NotFound = () => <h2>404 - Không tìm thấy trang</h2>;
function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          {/* MainLayout bọc tất cả các route con bên trong */}
          <Route path="/" element={<MainLayout />}>

            {/* Các route công khai (Public) */}
            <Route index element={<Home />} />
            <Route path="products" element={<Products />} />

            {/* Dynamic Route: :id là tham số động */}
            <Route path="products/:id" element={<ProductDetail />} />

            <Route path="login" element={<Login />} />
            {/* Private/Protected Route */}
            <Route
              path="dashboard"
              element={
                <ProtectedRoute>
                  <Dashboard />
                </ProtectedRoute>
              }
            />
            {/* Bắt các URL không tồn tại */}
            <Route path="*" element={<NotFound />} />

          </Route>
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}
export default App;
```

Hướng dẫn chạy kiểm thử & Thay đổi ngữ cảnh

Chạy lệnh `npm start` để khởi động server (thường ở `http://localhost:3000`).

1. Kiểm thử Nested Routes:

- **Hành động:** Nhấp qua lại giữa các menu (Trang chủ, Sản phẩm).
- **Kết quả:** Thanh điều hướng (Navbar) luôn giữ nguyên không bị load lại, chỉ có phần nội dung bên dưới (phần <Outlet/>) thay đổi.

2. Kiểm thử Dynamic Routes:

- **Hành động:** Vào menu "Sản phẩm", nhấp vào "Khóa học ReactJS".
- **Kết quả:** URL chuyển thành /products/1. Giao diện hiển thị "Sản phẩm có ID: 1".
- **Thay đổi ngữ cảnh:** Lên trực tiếp thanh URL của trình duyệt, gõ tay thay đổi số thành http://localhost:3000/products/999 và Enter.
- **Kết quả:** Màn hình sẽ tự động bắt lấy thông tin và hiển thị "Sản phẩm có ID: 999" (minh chứng cho sự linh hoạt của useParams).

3. Kiểm thử Protected Routes & Flow Đăng nhập:

- **Ngữ cảnh 1 (Chưa đăng nhập):** Đang ở Trang chủ, nhấp thẳng vào menu "Dashboard (Private)".
- **Kết quả:** Component ProtectedRoute phát hiện isAuthenticated === false, lập tức chặn lại và đá (redirect) người dùng về /login.
- **Ngữ cảnh 2 (Thay đổi State để Đăng nhập):** Tại trang Login, nhấn nút "Đăng nhập (Mock)".
- **Kết quả:** State isAuthenticated chuyển thành true. Hệ thống nhớ bạn vừa cố gắng vào Dashboard nên tự động điều hướng bạn thẳng vào lại /dashboard. Nút "Đăng xuất" màu đỏ cũng sẽ xuất hiện trên Navbar.
- **Ngữ cảnh 3 (Đăng xuất):** Nhấp nút "Đăng xuất" trên góc phải. State trở lại false. Nếu bạn đang đứng ở /dashboard và cố F5 (tải lại trang), bạn sẽ lại bị văng ra trang Login.